
EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

OPERATION IRON GATE - Instructor Solution Key

ARM Cortex-M33 • GDB • Ghidra

September 21, 2026

For educational and defensive security research only

The task and criterion headings in this key are word-for-word identical to `ACT-II-R.md`, so a student can match each criterion one to one.

Artifact Identity

The instructor-issued artifact hashes are:

```
ACT-II.bin          da216cff3f569983dcf7d5e14d192c23d491ea8126abee8d4c153c58f8d5a282
ACT-II.uf2          ad146617c91d96445df0ebbad4c9e282c2cdc011de985a343323d3df107856ec
ACT-II_fixed.bin    3deba303f5daa5b48a6fda2c583cff76916ea7012a7b2bbb8f9ea4b71372d304
ACT-II_fixed.uf2    d8789dd83098b421ea669015873bb499ccb39c2571ecc54f51b30f6e7b2bf7fe
```

Machine check: `python scripts/verify_ctf.py` returns 10/10 checks passed against the shipped and corrected images. It asserts the four byte pairs, that only those four offsets differ, and the `ACT-II.bin` and `ACT-II_fixed.bin` SHA-256 values. Both images are 51,160 bytes.

Task 1: Setup and Initial Analysis (10 points)

Solution

Ghidra Setup. Import `ACT-II.bin` as Raw Binary, language ARM Cortex 32 little endian default, base address `0x10000000`, then run auto-analysis. The Ghidra project name is `IronGate_Investigation`. Because every defect is a same-size in-place byte patch, the file offset and the VA differ by exactly `0x10000000` (`VA = offset + 0x10000000`).

Vector Table Decoding. First 32 bytes of `ACT-II.bin`:

```
00 20 08 20  5D 01 00 10  1B 01 00 10  1D 01 00 10
11 01 00 10  11 01 00 10  11 01 00 10  11 01 00 10
```

Evidence	Answer
Vector table base	<code>0x10000000</code>
Initial SP	<code>0x20082000</code>
Reset handler (as stored)	<code>0x1000015D</code>
Reset instruction address	<code>0x1000015C</code>

The stored reset handler address has bit 0 set, selecting Thumb mode. Clearing bit 0 gives the real entry `0x1000015C`.

Entry and Monitor Loop. From `ACT-II-main-disasm.txt`:

```

10000234 <main>:
10000234:    b508          push    {r3, lr}
10000236:    f003 fab7     bl      100037a8 <stdio_init_all>
1000023a:    4807          ldr     r0, [pc, #28] @ (10000258 <main+0x24>)
1000023c:    f003 fafe     bl      1000383c <__wrap_puts>
10000240:    f006 fb0a     bl      10006858 <monitor_init>
10000244:    b110          cbz     r0, 1000024c <main+0x18>
10000246:    f006 fbc5     bl      100069d4 <monitor_step>
1000024a:    e7fc          b.n     10000246 <main+0x12>

```

Element	Address
main	0x10000234
monitor_init	0x10006858
monitor_step	0x100069D4

Module Map. Anchors for the stripped image:

Module	Anchor function	Address
Entry	main	0x10000234
Random source	get_rand_32	0x100066FC
Desk receive path	monitor_rx_tick	0x10006704
State machine loop	monitor_step	0x100069D4
Interlock	sensor_read	0x10007080
Radio	radio_send_frame	0x100077C0
Annunciator	status_led_show	0x10007ABC
Deadbolt	servo_lock	0x10007BC0
Deadbolt	servo_unlock	0x10007BDC
Infrared keypad	ir_remote_poll	0x10007CA4
Keypad	keypad_push_digit	0x10007D38
Keypad	keypad_clear	0x10007D6C
Keypad	keypad_enter	0x10007D80
Keypad	keypad_get_pin	0x10007DA0
Authorization	auth_begin_request	0x10007E68
Authorization	auth_state_ok	0x10007E80
Authorization	auth_apply_grant	0x10007F2C
Crypto	crypto_aead_open	0x1000809C
KDF	crypto_kdf_argon2id	0x10008150

Module	Anchor function	Address
Envelope	envelope_seal_hex	0x100081C8
Envelope	envelope_open_hex	0x10008288

The four sabotage sites (summary):

Defect	Function	File offset	VA	Compromised	Correct
1 Replay	auth_apply_grant	0x7F43	0x10007F43	0xD2	0xD3
2 Forge	crypto_aead_open	0x811D	0x1000811D	0xD1	0xD0
3 State	inlined monitor_grant_release in monitor_rx_tick	0x6797	0x10006797	0xD1	0xD0
4 Fail-open	inlined monitor_fail_mode in monitor_step	0x6BCB	0x10006BCB	0xB9	0xB1

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Ghidra project created with the correct name and settings	2	Project <code>IronGate_Investigation</code> , raw binary import
[DOCUMENT] Processor configured as ARM Cortex 32 little endian default	2	Screenshot shows the correct processor
[DOCUMENT] Base address set to 0x10000000	2	Base <code>0x10000000</code>
[DOCUMENT] Vector table, initial stack pointer, and reset handler identified	2	Base <code>0x10000000</code> , initial SP <code>0x20082000</code> , reset handler <code>0x1000015D</code>
[DOCUMENT] main and the monitor loop (monitor_step) addresses identified	1	main <code>0x10000234</code> , monitor_step <code>0x100069D4</code>
[DOCUMENT] Module map identifies the keypad, auth, servo deadbolt, LEDs, DHT11 interlock, LCD access log, REX button, and sealed LoRa anchors	1	At least one correct anchor per module

Instructor Notes & Assembly

- Confirm the Ghidra import used `Raw Binary` , `ARM Cortex 32 little endian default` , base `0x10000000` , and that auto-analysis completed before any address was read. In the language dialog the student must search `Cortex` and pick the ARM Cortex 32 little endian default entry.
- Accept either the Import Results Summary or the Program Information window as proof of the name, language, and base address.
- The stored reset handler `0x1000015D` is odd because bit 0 selects Thumb; clearing it gives `0x1000015C` .
- Always say `reset handler` , never `reset pointer` .
- The module map is graded on coverage, not on exhaustive function recovery: one correctly named anchor per module is sufficient.

Task 2: Bug #1 Replay (20 points)

Solution

Locate the branch. In `auth_apply_grant` (starts at `0x10007F2C`) the anti-replay branch is at file offset `0x7F43` (VA `0x10007F43`). The corrected image is:

```

10007f2c <auth_apply_grant>:
10007f2c:      2800          cmp     r0, #0
10007f2e:      d04a          beq.n   10007fc6 <auth_apply_grant+0x9a>
10007f30:      e92d 41f0      stmdb   sp!, {r4, r5, r6, r7, r8, lr}
10007f34:      4616          mov     r6, r2
10007f36:      b094          sub     sp, #80 @ 0x50
10007f38:      b122          cbz     r2, 10007f44 <auth_apply_grant+0x18>
10007f3a:      6883          ldr     r3, [r0, #8]
10007f3c:      4604          mov     r4, r0
10007f3e:      428b          cmp     r3, r1
10007f40:      460d          mov     r5, r1
10007f42:      d303          bcc.n   10007f4c <auth_apply_grant+0x20>
10007f44:      2000          movs    r0, #0
10007f46:      b014          add     sp, #80 @ 0x50
10007f48:      e8bd 81f0      ldmia.w sp!, {r4, r5, r6, r7, r8, pc}
10007f4c:      4b1f          ldr     r3, [pc, #124] @ (10007fcc <auth_apply_grant+0xa0>)
10007f4e:      781b          ldrb    r3, [r3, #0]
10007f50:      2b00          cmp     r3, #0
10007f52:      d0f7          beq.n   10007f44 <auth_apply_grant+0x18>

```

Instruction decode. `ldr r3, [r0, #8]` loads the record's `last_seq`, `cmp r3, r1` compares it with the grant seq in `r1`, and the branch at `0x10007F42` decides admission. The correct condition is "accept only when the grant is strictly newer", so the branch must be `bcc.n` to `0x10007F4C` (taken when `last_seq < seq`, unsigned). The condition byte is the high byte of the halfword at `0x10007F42`, which is `0xD3` for `bcc.n` and `0xD2` for `bcs.n`.

Address	File offset	Compromised byte	Compromised instruction	Correct byte	Correct instruction
<code>0x10007F43</code>	<code>0x7F43</code>	<code>0xD2</code>	<code>bcs.n 0x10007F4C</code>	<code>0xD3</code>	<code>bcc.n 0x10007F4C</code>

Patch.

File Offset	VA	Original Bytes	Patched Bytes
<code>0x7F43</code>	<code>0x10007F43</code>	<code>03 D2</code>	<code>03 D3</code>

Why a captured grant now fails. Under the compromised `bcs.n`, the branch to the accept path is taken when `last_seq >= seq`, so an old or equal sequence is admitted and a captured GRANTED frame reopens the door. After the patch, `bcc.n` takes the accept path only when `last_seq < seq`; anything else falls through to `0x10007F44` and returns 0. The anti-replay window is strictly monotonic again, and a replay lands on DENIED without moving the bolt.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Located the anti-replay branch at <code>0x10007F43</code>	5	Address and function identified

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Documented bcc.n versus bcs.n and the seq > last_seq rule	5	Correct branch semantics and strict-greater rule
[DOCUMENT & PATCH] Patched 0xD2 to 0xD3 so the anti-replay window is restored	7	Byte 0xD2 changed to 0xD3
[DOCUMENT] Explained that a captured GRANTED frame then fails to reopen the door	3	Freshness tracked by a strictly monotonic sequence

Instructor Notes & Assembly

- The condition byte is the high byte at 0x7F43 ; the listing shows the halfword d303 and the on-disk bytes are 03 D3 (little-endian).
- cmp r3, r1 is an unsigned unsigned compare of last_seq (r3) against seq (r1). bcc means carry clear, so last_seq < seq .
- A common error is reversing the explanation. Under the compromise the accept path is taken when the sequence is not strictly greater.
- Full credit requires both the byte change and a correct statement of the strict-greater rule.
- Note that the tag check later in the function is still correct; replay is a freshness failure, not an authenticity failure.

Task 3: Bug #2 Forge (20 points)

Solution

Locate the branch. In crypto_aead_open (starts at 0x1000809C) the tag comparison folds the four 32-bit XOR words with OR into r3 , then masks and tests the low byte. The branch is at file offset 0x811D (VA 0x1000811D). The corrected image is:

```

10008118:    f013 03ff    ands.w  r3, r3, #255    @ 0xff
1000811c:    d003        beq.n   10008126 <crypto_aead_open+0x8a>
1000811e:    4628        mov     r0, r5
10008120:    b023        add     sp, #140        @ 0x8c
10008122:    e8bd 8ff0    ldmia.w sp!, {r4, r5, r6, r7, r8, r9, sl, fp, pc}
10008126:    9701        str     r7, [sp, #4]
10008128:    9303        str     r3, [sp, #12]
1000812a:    6920        ldr     r0, [r4, #16]
1000812c:    6961        ldr     r1, [r4, #20]
1000812e:    ab04        add     r3, sp, #16
10008130:    c303        stmia  r3!, {r0, r1}
10008132:    992f        ldr     r1, [sp, #188] @ 0xbc
10008134:    a80a        add     r0, sp, #40    @ 0x28
10008136:    9100        str     r1, [sp, #0]
10008138:    2201        movs    r2, #1
1000813a:    9b2c        ldr     r3, [sp, #176] @ 0xb0
1000813c:    a903        add     r1, sp, #12
1000813e:    f000 fbbb    bl      100088b8 <chacha20_xor>
10008142:    2001        movs    r0, #1
10008144:    b023        add     sp, #140        @ 0x8c
10008146:    e8bd 8ff0    ldmia.w sp!, {r4, r5, r6, r7, r8, r9, sl, fp, pc}

```

Instruction decode. The compare is a constant-time tag check: the computed tag is XORed against the received tag word by word, the differences are ORed together, and the result is masked. A zero result means the tags match. The correct code accepts only a zero difference, so the branch at `0x1000811C` must be `beq.n (0xD0)` to the decrypt path at `0x10008126`. The condition byte is the high byte at `0x1000811D`.

Address	File offset	Compromised byte	Compromised instruction	Correct byte	Correct instruction
<code>0x1000811D</code>	<code>0x811D</code>	<code>0xD1</code>	<code>bne.n 0x10008126</code>	<code>0xD0</code>	<code>beq.n 0x10008126</code>

Patch.

File Offset	VA	Original Bytes	Patched Bytes
<code>0x811D</code>	<code>0x1000811D</code>	<code>03 D1</code>	<code>03 D0</code>

Why forged grants now fail. Under the compromised `bne.n`, a non-zero tag difference branches to the decrypt path at `0x10008126`, so a forged frame is decrypted and returns 1, while a genuine frame with a zero difference falls through to return 0 and is rejected. After the patch, only a zero difference reaches `0x10008126`, so a forged grant returns 0 and the gate denies.

The constant-time construction never returns early on the first differing byte; it always ORs all word differences, so timing does not reveal how many bytes matched.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Located the tag-difference branch at <code>0x1000811D</code>	5	Address and function identified
[DOCUMENT] Documented the constant-time tag compare and the branch condition	5	Zero difference means accept
[DOCUMENT & PATCH] Patched <code>0xD1</code> to <code>0xD0</code> so forged grants fail authentication	7	Byte <code>0xD1</code> changed to <code>0xD0</code>
[DOCUMENT] Explained why a non-zero Poly1305 tag difference must be rejected	3	A non-zero difference means forged or corrupt

Instructor Notes & Assembly

- The branch condition byte is the high byte at `0x811D`; the halfword is `d003` for `beq.n` and `d103` for `bne.n`, and the file bytes are `03 D0` for the fix and `03 D1` for the compromise.
- The compare is a full word-wise OR of the four tag-difference words, folded into a single zero test at `0x10007FD0`.
- Full credit requires the inversion explanation: the compromised build accepts a non-zero difference and rejects a zero difference.
- Point out that the rest of the AEAD is correct; only this seam was broken.

Task 4: Bug #3 State (20 points)

Solution

Locate the branch. The `monitor_grant_release` path is inlined into `monitor_rx_tick` (starts at `0x10006704`). After `auth_apply_grant` returns true, the code calls `auth_state_ok(&g_auth)` and branches at file offset `0x6797` (VA `0x10006797`). The corrected image is:

```

10006780:    4662      mov     r2, ip
10006782:    482b      ldr     r0, [pc, #172] @ (10006830 <monitor_rx_tick+0x12c>)
10006784:    6829      ldr     r1, [r5, #0]
10006786:    f001 fbd1 bl      10007f2c <auth_apply_grant>
1000678a:    2800      cmp     r0, #0
1000678c:    d0d8      beq.n   10006740 <monitor_rx_tick+0x3c>
1000678e:    4828      ldr     r0, [pc, #160] @ (10006830 <monitor_rx_tick+0x12c>)
10006790:    f001 fb76 bl      10007e80 <auth_state_ok>
10006794:    2800      cmp     r0, #0
10006796:    d0d3      beq.n   10006740 <monitor_rx_tick+0x3c>
10006798:    4628      mov     r0, r5
1000679a:    f000 fc71 bl      10007080 <sensor_read>
...
10006740:    4c39      ldr     r4, [pc, #228] @ (10006828 <monitor_rx_tick+0x124>)
10006742:    f001 fa3d bl      10007bc0 <servo_lock>
10006746:    2001      movs    r0, #1
10006748:    7020      strb    r0, [r4, #0]
1000674a:    f001 f9b7 bl      10007abc <status_led_show>

```

Instruction decode. `auth_state_ok` recomputes the state tag over the authorization record and compares it in constant time. `cmp r0, #0` tests the verdict. The correct code denies when the state tag does not match, so the branch at `0x10006796` must be `beq.n (0xD0)` to the deny path at `0x10006740`, which seals the bolt. Only a true result falls through to the sensor interlock and `servo_unlock`. The condition byte is the high byte at `0x10006797`.

Address	File offset	Compromised byte	Compromised instruction	Correct byte	Correct instruction
<code>0x10006797</code>	<code>0x6797</code>	<code>0xD1</code>	<code>bne.n 0x10006740</code>	<code>0xD0</code>	<code>beq.n 0x10006740</code>

Patch.

File Offset	VA	Original Bytes	Patched Bytes
<code>0x6797</code>	<code>0x10006797</code>	D3 D1	D3 D0

Why the flipped verdict now fails. Under the compromised `bne.n`, a valid `auth_state_ok` result branches to the deny path and an invalid result falls through to the release path, so the authenticated-state check is inverted and effectively bypassed. A debugger that sets `g_auth.granted = 1` without recomputing the state tag changes the record, so `auth_state_ok` returns false; under the compromise that false result falls through and opens the door. After the patch, the false result at `0x10006796` branches to `0x10006740`, seals the bolt, and announces DENIED before the bolt can move.

GDB demonstration. The release path is inlined, so break at the instruction that loads `&g_auth` for the state check, `0x1000678E`, and tamper with the verdict. No register values are fabricated here; the evidence is the command sequence and the code path the target takes.

```

arm-none-eabi-gdb ACT-II.elf
(gdb) target extended-remote localhost:3333
(gdb) monitor reset halt
(gdb) break *0x1000678E
(gdb) continue
(gdb) p g_auth
(gdb) set g_auth.granted = 1
(gdb) set {unsigned char}0x20013334 = 1
(gdb) continue

```

Expected observations:

- `g_auth` lives at `0x20013334`. The `granted` boolean is byte 0; both forms above set it.
- On the corrected image, `auth_state_ok` recomputes the state tag over the modified record, finds that `granted` no longer matches the tag, returns false, and the `beq.n` at `0x10006796` sends control to `0x10006740`, so the bolt is sealed and the red DENIED lamp is lit.
- On the compromised image, the `bne.n` at `0x10006796` misses the deny path, control falls through to `0x10006798`, the interlock passes, and the green GRANTED lamp is lit as `servo_unlock` runs.

To reach `0x1000678E`, drive a legitimate grant first so `auth_apply_grant` returns true, or break on the call at `0x10006786` and step to the state check.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Located the authenticated-state branch at <code>0x10006797</code>	5	Address and inlined release path identified
[DOCUMENT] Documented the <code>auth_state_ok</code> gate and the branch condition	5	Deny when the state tag does not match
[DOCUMENT & PATCH] Patched <code>0xD1</code> to <code>0xD0</code> so the authenticated-state check is restored	7	Byte <code>0xD1</code> changed to <code>0xD0</code>
[DOCUMENT] Demonstrated with GDB that <code>g_auth.granted = 1</code> without recomputing the tag no longer opens the door	3	Command sequence and the correct observed rejection

Instructor Notes & Assembly

- The release path is inlined into `monitor_rx_tick`; there is no standalone `monitor_grant_release` symbol in the stripped image.
- `g_auth` is at `0x20013334` and its first byte is `granted`. The record is ten bytes: `granted`, `pending`, `seq[4]`, `last_seq[4]`.
- The `auth_state_ok` call is at `0x10006790`; the check is at `0x10006796` and the changed byte is the high byte at `0x10006797`.
- This is the TOCTOU lesson: the wire is authenticated, then a plain mutable boolean is trusted. The fix is to authenticate the state, not to add more cipher.
- Grade the GDB point on a real command sequence and the correct observed code path, not on a memorized register dump.

Task 5: Bug #4 Fail-Open (20 points)

Solution

Locate the branch. The `monitor_fail_mode` path is inlined into `monitor_step` (starts at `0x100069D4`). In the PENDING timeout path the code tests the fail-mode policy and branches at file offset `0x6BCB` (VA `0x10006BCB`). The corrected image is:

```

10006bc2:    f4ff af6b    bcc.w  10006a9c <monitor_step+0xc8>
10006bc6:    4b50        ldr     r3, [pc, #320] @ (10006d08 <monitor_step+0x334>)
10006bc8:    781b        ldrb    r3, [r3, #0]
10006bca:    b16b        cbz     r3, 10006be8 <monitor_step+0x214>
10006bcc:    f000 fff8    bl     10007bc0 <servo_lock>
10006bd0:    2001        movs    r0, #1
10006bd2:    7028        strb    r0, [r5, #0]
10006bd4:    f000 ff72    bl     10007abc <status_led_show>
...
10006be8:    f000 fff8    bl     10007bdc <servo_unlock>
10006bec:    2003        movs    r0, #3
10006bee:    7028        strb    r0, [r5, #0]
10006bf0:    f000 ff64    bl     10007abc <status_led_show>

```

Instruction decode. Once `now >= g_pending_until_us`, the code loads the fail-mode policy byte (`g_fail_secure` at SRAM `0x20001254`) and tests it with `cbz`, which branches when the policy is zero. The correct ingress policy is fail secure: when fail-secure is enabled (non-zero), the code must fall through to `servo_lock` and announce DENIED; the branch to the unlock path at `0x10006BE8` is taken only when the policy is zero. The condition byte is the high byte at `0x10006BCB`.

Address	File offset	Compromised byte	Compromised instruction	Correct byte	Correct instruction
<code>0x10006BCB</code>	<code>0x6BCB</code>	<code>0xB9</code>	<code>cbnz r3, 0x10006BE8</code>	<code>0xB1</code>	<code>cbz r3, 0x10006BE8</code>

Patch.

File Offset	VA	Original Bytes	Patched Bytes
<code>0x6BCB</code>	<code>0x10006BCB</code>	6B B9	6B B1

Why the timeout now stays locked. Under the compromised `cbnz`, the branch to `servo_unlock` is taken exactly when fail-secure is enabled, so an authorization timeout opens the bolt (fail-open). After the patch, `cbz` branches to the unlock path only when the policy is zero; with fail-secure enabled the code falls through to `servo_lock`, announces DENIED, and the bolt stays shut. A grant that never arrives leaves the gate locked.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Located the fail-mode branch at <code>0x10006BCB</code>	5	Address and inlined fail-mode path identified
[DOCUMENT] Documented fail-secure versus fail-open on an authorization timeout	5	Correct policy and branch semantics

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT & PATCH] Patched 0xB9 to 0xB1 so a timeout leaves the bolt locked	7	Byte 0xB9 changed to 0xB1
[DOCUMENT] Explained that an authorization timeout must fail secure	3	The bolt stays locked when no grant arrives

Instructor Notes & Assembly

- The fail-mode path is inlined into `monitor_step`; there is no standalone `monitor_fail_mode` symbol in the stripped image.
- The branch is at `0x10006BCA` (b16b fixed); the changed byte is the high byte at `0x10006BCB`.
- The unlock path is `0x10006BE8`; the lock path is the fall-through at `0x10006BCC`.
- Full credit requires both the byte change and the policy explanation: fail-secure is the correct default for security ingress, while the REX egress path on GP15 is deliberately fail-safe and unauthenticated for life safety. The two halves of one door can need opposite defaults.

Task 6: Export and Verify (10 points)

Solution

Export. In Ghidra, `File -> Export Program...`, choose `Binary Format`, and save as `ACT-II_fixed.bin`. The shipped image is 51,160 bytes.

Convert.

```
python uf2conv.py ACT-II_fixed.bin --base 0x10000000 --family 0xe48bff59 --output ACT-II_fixed.uf2
```

If `uf2conv.py` is not in the working directory, use the copy shipped with the project repository. The UF2 for ACT-II is 102,912 bytes.

Verify.

```
python scripts/verify_ctf.py
```

Expected result:

```
10/10 checks passed
```

Hardware proof. Flash `ACT-II_fixed.uf2` in BOOTSEL mode and confirm:

- a replay of a captured GRANTED frame lands on DENIED and the bolt does not move;
- a forged grant fails the tag and lands on DENIED;
- a debugger write to `g_auth.granted` is rejected by the state tag before the bolt moves;
- an authorization timeout leaves the bolt locked and the red lamp lit;
- a legitimate grant still opens the deadbolt for the bounded hold, and REX still opens it for life safety.

Summary of all patches.

#	Bug	File Offset	Flash Address	Original Byte	Patched Byte
1	Replay	0x7F43	0x10007F43	D2	D3
2	Forge	0x811D	0x1000811D	D1	D0
3	State	0x6797	0x10006797	D1	D0
4	Fail-open	0x6BCB	0x10006BCB	D1	D0

Reflection mapping. The four defects map to real access-control failures:

Defect	Real-world failure
Replay	A captured valid command works twice, so an old credential reopens the door.
Forge	A broken tag check lets a forged unlock command look authentic.
State	A door that trusts a plain verdict boolean opens for anyone with a debugger.
Fail-open	A timeout that unlocks turns a lost link into an open door.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[PATCH] Exported ACT-II_fixed.bin from Ghidra	2	Valid patched binary
[PATCH] Converted to ACT-II_fixed.uf2 with the correct base and family	2	--base 0x10000000 --family 0xe48bff59
[DOCUMENT] scripts/verify_ctf.py passes and hardware proves the correct behavior	3	Verifier passes and the hardware proof is shown
[DOCUMENT] Reflection maps each of the four defects to a real-world access-control failure	3	Specific mapping for all four

Instructor Notes & Assembly

- Confirm the exported image differs from ACT-II.bin in exactly the four bytes in the table; scripts/verify_ctf.py checks this and the SHA-256 values.
- Confirm the UF2 conversion used base 0x10000000 and family 0xe48bff59.
- The shipped image is 51,160 bytes; the corrected image must be the same size because every patch is in place.
- Grade the reflection on specificity, not length: each of the four defects should name a concrete access-control consequence.

How To Breadboard

Device	Pin on device	Pico 2 GPIO	Notes
DHT11 data	DATA	GP4	10 kOhm pull-up to 3.3 V if the module needs it

Device	Pin on device	Pico 2 GPIO	Notes
1602 LCD	SDA	GP2	I2C1, backpack address 0x27
1602 LCD	SCL	GP3	I2C1, 100 kHz
1602 LCD	VCC / GND	VBUS 5 V / GND	The backpack needs 5 V, not 3.3 V
RYLR998	RX	GP8 (Pico TX)	UART1, 115200, network ID 18
RYLR998	TX	GP9 (Pico RX)	UART1
IR receiver	OUT	GP5	VS1838B, internal pull-up enabled
Servo	signal	GP14	PWM 50 Hz; 1000 uF bulk cap across servo 5 V and GND
Red LED	anode	GP16	220 to 330 ohm to GND
Yellow LED	anode	GP17	220 to 330 ohm to GND
Green LED	anode	GP18	220 to 330 ohm to GND
REX button	leg 1	GP15	Internal pull-up; leg 2 to GND, never to 3.3 V
Onboard LED	built in	GP25	Heartbeat
Debug Probe	SWCLK / SWDIO / GND	debug header	For GDB only

Use 3.3 V logic on every GPIO. The only 5 V connection is the LCD backpack supply. Keep the 1000 uF capacitor on the servo rail to absorb the SG90 current spike.

Complete Grading Summary

Task	Title	Points
Task 1	Setup and Initial Analysis	10
Task 2	Bug #1 Replay	20
Task 3	Bug #2 Forge	20
Task 4	Bug #3 State	20
Task 5	Bug #4 Fail-Open	20
Task 6	Export and Verify	10
TOTAL		100

Instructor Notes

Safety: Use only the supplied Pico 2, Debug Probe, and firmware. Never connect the exercise to an operational access-control system, a pharmaceutical network, a public network, a military system, or a third-party device.

Common Student Mistakes

- Patching the low byte of the branch at `0x7F42` instead of the condition byte at `0x7F43`.
- Reversing the replay explanation: under the compromise the accept path is taken when the sequence is not strictly greater.
- Reading the tag branch backwards and believing the compromised build accepts genuine frames.
- Searching for a standalone `monitor_grant_release` or `monitor_fail_mode` symbol and missing that both are inlined into their callers.
- Treating the SRAM verdict as a crypto problem instead of a state-integrity problem.
- Confusing fail-secure and fail-open, or trying to close the REX egress path.
- Forgetting the UF2 conversion or using the wrong family flag.
- Fabricating a GDB session instead of showing the command sequence and the real code path.

Partial Credit Guidelines

- Award partial credit for a correct address without the correct byte, or a correct byte without the address.
- Award partial credit for documented before/after bytes without the control-flow explanation, or vice versa.
- Award partial credit for a correct GDB command sequence without a clear statement of the observed denial, or the denial without the commands.
- Award partial credit for a correct fail-mode policy statement without the byte change, or the byte change without the policy.
- Award no credit for patches that alter any byte outside the four documented offsets, and no credit for a fabricated GDB session.

Appendix: Expected Binary Diff

These offsets are from the compiled image loaded at `0x10000000`.

```

--- ACT-II.bin (compromised)
+++ ACT-II_fixed.bin (corrected)

Offset 0x00007F43: D2 -> D3 (bcs.n 0x10007F4C -> bcc.n 0x10007F4C)
Offset 0x0000811D: D1 -> D0 (bne.n 0x10008126 -> beq.n 0x10008126)
Offset 0x00006797: D1 -> D0 (bne.n 0x10006740 -> beq.n 0x10006740)
Offset 0x00006BCB: B9 -> B1 (cbnz r3, 0x10006BE8 -> cbz r3, 0x10006BE8)

```

#	Bug	File Offset	Flash Address	Original Bytes	Patched Bytes
1	Replay	<code>0x7F43</code>	<code>0x10007F43</code>	<code>03 D2</code>	<code>03 D3</code>
2	Forge	<code>0x811D</code>	<code>0x1000811D</code>	<code>03 D1</code>	<code>03 D0</code>
3	State	<code>0x6797</code>	<code>0x10006797</code>	<code>D3 D1</code>	<code>D3 D0</code>
4	Fail-open	<code>0x6BCB</code>	<code>0x10006BCB</code>	<code>6B B9</code>	<code>6B B1</code>

Four defects, four changed bytes in four instructions: the replay condition, the tag-compare condition, the authenticated-state condition, and the fail-mode condition. No other byte in either image differs.